

A Memory-Efficient Classification Pipeline for an Embedded Sensor

Pina Cammisano

September 2026

1 Proposed Pipeline

I propose a classification pipeline for an embedded device where the training is done offline and only a small lookup model is stored on the device.

During the offline stage, DBSCAN is used to discover classes from unlabeled data. A Gaussian Kernel Density Estimate (KDE) is then fitted to each discovered class. The KDE surfaces are normalized, sampled onto a fixed grid, and stored as unsigned 8-bit values.

The device only performs the online part of the pipeline. It does not need the original training data or need to calculate a KDE for every query. It only needs the stored byte tables, the scaling bounds, the grid size, and a reject threshold.

A stored byte is converted back to a normalized density value using

$$\tilde{g}_c(\mathbf{x}_{ij}) = \frac{q_{ij}}{255}. \quad (1)$$

I would use nearest-cell lookup for the final design because it requires only one byte read per class. Bilinear lookup can make the result smoother near a grid boundary, but it requires four reads per class.

Algorithm 1 shows the online classification procedure.

2 Application

A possible application for this pipeline is a battery-powered vibration monitoring sensor mounted on a small electric motor. The device could measure two vibration features and classify different operating patterns directly on the sensor.

For this example, I assume that about 4 kB of flash memory is available for the classification tables. A small lookup model is more practical than

Algorithm 1 Table-Lookup Classification

Require: Raw query point (x, y)

Require: One byte table Q_c per class

Require: Training bounds $(x_{\min}, x_{\max})$ and $(y_{\min}, y_{\max})$

Require: Grid size G and reject threshold τ

Ensure: Predicted class or UNKNOWN

```

1:  $\tilde{x} \leftarrow \frac{x - x_{\min}}{x_{\max} - x_{\min}}$ 
2:  $\tilde{y} \leftarrow \frac{y - y_{\min}}{y_{\max} - y_{\min}}$ 
3:  $\text{col} \leftarrow \lfloor \tilde{x}G \rfloor$ 
4:  $\text{row} \leftarrow \lfloor \tilde{y}G \rfloor$ 
5: Clip row and column to 0 through  $G - 1$ 
6: for each class  $c$  do
7:    $v_c \leftarrow Q_c[\text{row}, \text{col}] / 255$ 
8: end for
9:  $\hat{c} \leftarrow \arg \max_c v_c$ 
10: if  $v_{\hat{c}} < \tau$  then
11:   return UNKNOWN
12: end if
13:  $p_c \leftarrow v_c / \sum_k v_k$ 
14: return  $\hat{c}$  and distribution  $p$ 

```

storing the original training data or performing KDE calculations for every new measurement.

Labeled training data may also not be available when the sensor is first installed. DBSCAN is useful in this situation because it can discover groups from the measurements without requiring known class labels.

The classes may also be curved or irregular instead of round, like the crescents and ring in this lab. This makes DBSCAN more suitable than k-means, which assigns each point according to its nearest cluster center.

The reject threshold is also important for this

Table 1: Effect of scaling on DBSCAN.

Features	$\epsilon = 0.06$	$\epsilon = 0.30$
Raw	0 clusters, 618 noise	10 clusters, 50 noise
Scaled	3 clusters, 33 noise	1 cluster, 0 noise

application. If a new measurement does not match any learned operating region, the sensor should report UNKNOWN instead of forcing the measurement into one of the known classes.

3 Method and Setup

The generated dataset contained 1030 two-dimensional samples. A fixed random seed of 20250903 was used.

The data was divided into 60% training, 20% validation, and 20% test. This produced 618 training points, 206 validation points, and 206 test points.

The training set was used to build the model. The validation set was used to choose the KDE bandwidth. The test set was kept separate and used only after the design was frozen so that the model was not tuned using the test results.

The features were scaled to the interval $[0, 1]$ before clustering.

The final DBSCAN parameters were

$$\epsilon = 0.06, \quad \text{min_samples} = 8.$$

DBSCAN found three clusters and 33 noise points. The cluster sizes were

$$209, \quad 198, \quad 178.$$

Table 1 shows how scaling affected DBSCAN.

With scaled data, $\epsilon = 0.03$ produced 15 clusters and 82 noise points. This value was too small and fragmented the shapes.

With scaled data, $\epsilon = 0.30$ produced one cluster and no noise. This value was too large and merged the shapes together.

Figure 1 is Diagram B from the lab and compares DBSCAN and k-means on the same training points.

A KDE was then fitted separately to each class discovered by DBSCAN. The bandwidth was selected using the validation set.

Table 2 shows the bandwidth sweep.

Table 2: Validation bandwidth sweep.

h	Mean log-likelihood
0.03	1.668
0.04	1.775
0.05	1.806
0.06	1.767
0.08	1.632
0.10	1.487
0.12	1.353

The highest validation score was obtained with

$$h = 0.05.$$

The final model therefore used

$$G = 16, \quad h = 0.05, \quad \tau = 0.12.$$

The stored class maps used unsigned 8-bit values.

4 Evidence

The results support using this pipeline because the non-convex classes were discovered without labels and the stored byte model kept the same decisions as the continuous KDE on the final test set.

Figure 2 is Diagram C and shows the continuous KDE decision surface.

Figure 3 is Diagram D and compares the continuous per-class surfaces with the stored uint8 surfaces.

The worst-case uint8 quantization error was

$$0.195\% \text{ of peak.}$$

Table 3 shows the storage needed by each format.

The uint8 table footprint is

$$\begin{aligned} \text{flash} &= K \cdot G^2 \cdot b \\ &= 3 \cdot 16^2 \cdot 1 \\ &= 768 \text{ bytes.} \end{aligned} \tag{2}$$

The four scaling bounds require another 16 bytes.

I would use the uint8 format because it reduces the three class tables from 6144 bytes with float64 to 768 bytes, while the measured quantization error is only 0.195%.

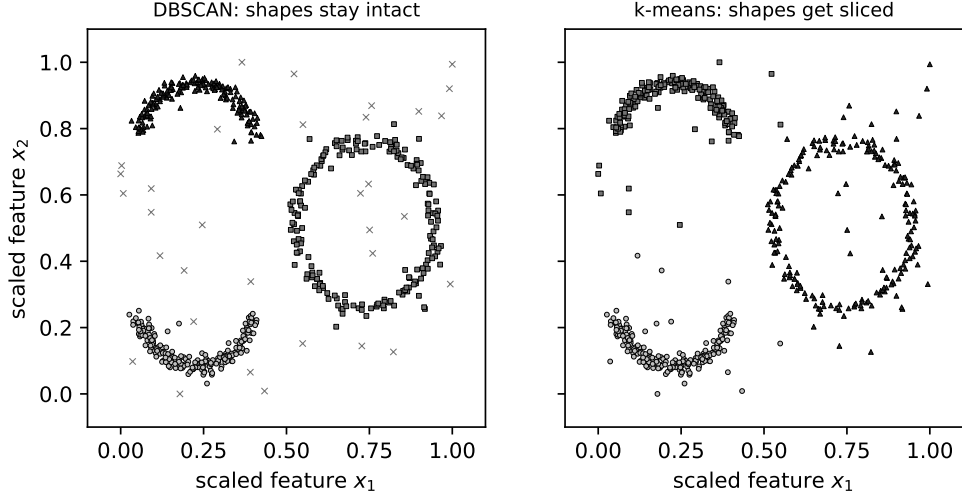


Figure 1: Diagram B — DBSCAN and k-means on the same training data. DBSCAN follows the non-convex shapes, while k-means divides the data using cluster centers and slices the shapes.

Table 3: Model storage using a 16×16 grid.

Storage	Bytes/cell	Per class	All 3 classes	Error
uint8	1	256 B	768 B	0.195%
uint16	2	512 B	1536 B	—
float32	4	1024 B	3072 B	—
float64	8	2048 B	6144 B	—

Changing the grid from 16×16 to 32×32 increased the three uint8 tables to

3072 bytes.

This is four times larger because the storage grows with G^2 .

Figure 4 is Diagram E and shows one held-out query classified using the stored model.

The raw query was

(5.9996, 3.1787)

and it mapped to row 11 and column 13.

Nearest-cell lookup produced

[0, 255, 0]

with the normalized distribution

[0%, 100%, 0%].

Bilinear lookup produced approximately

[0, 221.8, 0]

and also gave

[0%, 100%, 0%].

Both methods selected class 1.

Nearest-cell lookup requires $K = 3$ byte reads for three classes. Bilinear lookup requires $4K = 12$ byte reads.

The 3×3 byte window around the query for class 1 was

$$\begin{bmatrix} 187 & 164 & 76 \\ 208 & 255 & 183 \\ 68 & 159 & 222 \end{bmatrix}.$$

The values change noticeably between nearby cells. This shows why bilinear lookup can smooth the decision near grid boundaries.

Continuous decision surface ($\tau = 0.12, h = 0.05$)

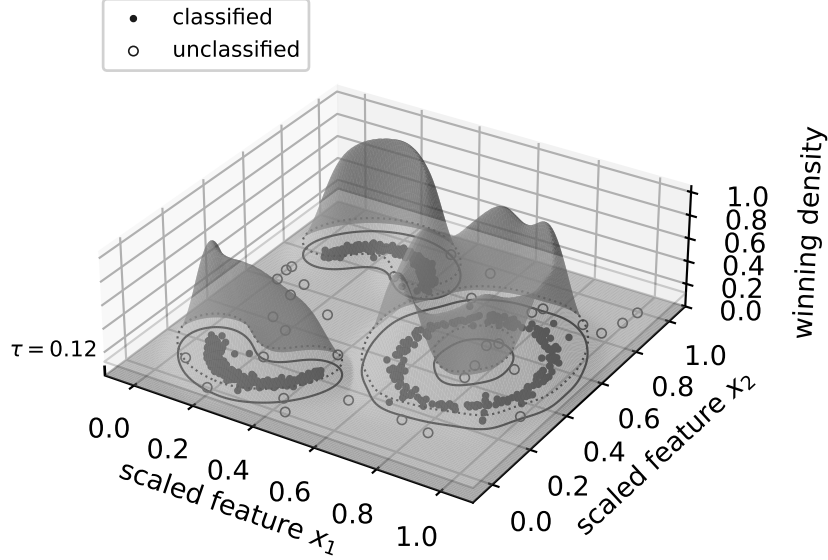


Figure 2: Diagram C — Continuous KDE decision surface with the data projected onto the feature plane.

The reject region contained

$$127/256 = 49.6\%$$

of the grid cells.

The point

$$(4.8, 1.6)$$

is located in the hollow center of the ring. The classifier returned UNKNOWN. This is the correct result because the point does not have a strong density for any class even though it is surrounded by the ring.

Without a reject threshold, the classifier would still be forced to choose one of the three classes.

The final test used all 206 held-out test points. The stored uint8 model agreed with the continuous KDE on

$$100.0\%$$

of the test points.

Table 4 shows the confusion table.

Table 4: Stored uint8 versus continuous KDE on the test set.

	c0	c1	c2	unk
class 0	69	0	0	0
class 1	0	68	0	0
class 2	0	0	63	0
unknown	0	0	0	6

There were no disagreements between the stored model and the continuous KDE on the held-out test set.

The results were 69 matching class 0 decisions, 68 matching class 1 decisions, 63 matching class 2 decisions, and 6 matching UNKNOWN decisions.

This means that the memory saving from the uint8 representation did not change any decisions on these 206 held-out test points.

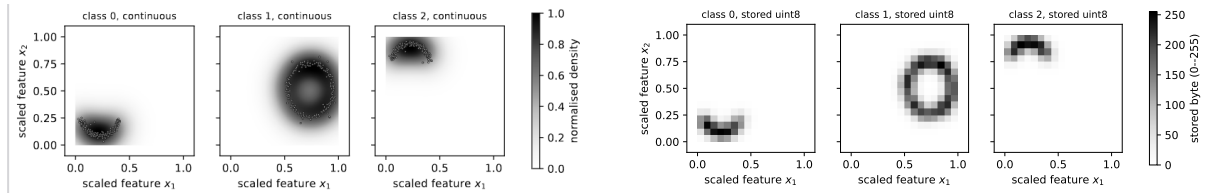


Figure 3: Diagram D — Continuous KDE class maps and their stored uint8 versions. The main class shapes remain visible after discretization and quantization.

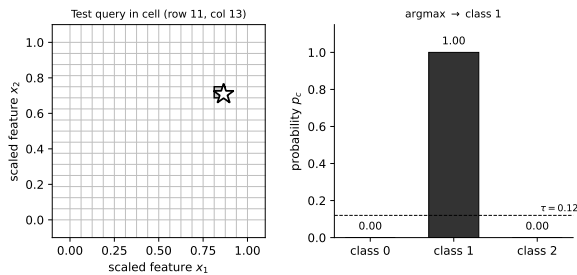


Figure 4: Diagram E — One held-out query classified using the stored grid. The winning class is selected using the largest stored class value.

5 Limitations and Remaining Uncertainty

There are several limitations to these results.

First, only one synthetic two-dimensional dataset was tested. Real sensor data could contain more noise, sensor drift, extra features, and different operating conditions.

Second, DBSCAN created the class labels. There was no ground-truth answer key. DBSCAN could merge two real classes into one cluster or split one real class into two clusters while the figures could still appear reasonable.

The 100% test result is also agreement with the continuous KDE and not accuracy against real class labels. It shows that the stored model reproduced the continuous model, but it does not prove that either model represents the correct real-world classes.

The grid approximation is a larger concern than the uint8 quantization. The quantization error was only 0.195%, while Figure 3 shows that the fixed grid loses some spatial detail compared with the continuous surfaces.

The ring-center result also shows something that

normal test accuracy would not reveal. Test samples taken from the three known shapes would not normally appear in the empty center of the ring. A classifier could therefore have a high test score while still giving a confident but incorrect answer in an empty region.

For a 4 kB table budget, a 16×16 uint8 grid requires 256 bytes per class. The number of class tables that could fit is

$$4096/256 = 16.$$

A 32×32 uint8 grid requires 1024 bytes per class, so only

$$4096/1024 = 4$$

class tables could fit in the same budget.

The larger grid gives better spatial resolution, but fewer classes can fit in memory.

Bilinear lookup would be useful if moving slightly across a grid boundary could cause an important control output to change suddenly. For example, the extra reads may be worth it in a machine-control application.

For a simple monitoring sensor that only records a state or sends an occasional alert, nearest-cell lookup is simpler and requires fewer reads.

Scaling was useful even though the two original features had similar ranges. It gives DBSCAN a consistent distance scale and maps both features directly into $[0, 1]$, which also makes the grid lookup simple.

The offline path could also become incorrect without changing the online algorithm. For example, future sensor data may drift away from the data used to build the DBSCAN clusters, or the KDE tables may no longer represent the current operating conditions.

The four scaling bounds are also important. If a new measurement falls outside the original training bounds, clipping it may incorrectly force it into an edge grid cell.

I would reject measurements that are clearly outside the training bounds. If this happens often, I would collect new data and re-fit the offline model and scaling bounds.

Bandwidth is another uncertainty. The sweep showed that $h = 0.03$ produced sharper maps, while $h = 0.12$ produced smoother maps and began filling the hollow center of the ring.

The next step would be to test this pipeline on real labeled sensor data. I would measure true classification accuracy, rejection rate, sensitivity to sensor drift, and whether the 16×16 nearest-cell lookup still gives enough spatial resolution.